

JAVA SpringBoot CODES

 UserServiceApplication.java

```
package com.example.users;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
        System.out.println("✅ User Service Application started successfully  
at http://localhost:8081/");
    }
}
```

 User.java (Model)

```
package com.example.users.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    private int id;
    private String name;
    private String email;
}
```

 UserService.java

```
package com.example.users.service;

import com.example.users.model.User;
import org.springframework.stereotype.Service;

import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

@Service
public class UserService {

    private final Map<Integer, User> users = new HashMap<>();
    private final AtomicInteger idCounter = new AtomicInteger(1); // Auto ID

    // Add a new user
    public User addUser(User user) {
        int id = idCounter.getAndIncrement();
        user.setId(id);
        users.put(id, user);
        return user;
    }

    // Get all users
    public List<User> getAllUsers() {
        return new ArrayList<>(users.values());
    }

    // Get a user by ID
    public User getUserById(int id) {
        return users.get(id);
    }

    // Delete a user by ID
    public void deleteUser(int id) {
        users.remove(id);
    }
}
```

```

package com.example.users.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;
import java.util.ArrayList;

@RestController
public class UserController {

    @GetMapping("/users")
    public List<String> getUsers() {
        // Dummy data
        List<String> users = new ArrayList<>();
        users.add("User1");
        users.add("User2");
        return users;
    }
}

```

 home.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>User Service Dashboard</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link href="https://fonts.googleapis.com/css2?
family=Poppins:wght@400;600&display=swap" rel="stylesheet">
    <style>
        body {
            font-family: 'Poppins', sans-serif;
            background: linear-gradient(135deg, #74ebd5, #9face6);
            color: #333;
            min-height: 100vh;
            display: flex;

```

```

        align-items: center;
        justify-content: center;
    }

    .card {
        background: white;
        border-radius: 20px;
        box-shadow: 0 10px 30px rgba(0,0,0,0.15);
        padding: 40px;
        max-width: 500px;
        text-align: center;
    }

    h1 {
        font-weight: 600;
        color: #4b00ff;
    }

    .btn-custom {
        background-color: #4b00ff;
        color: white;
        padding: 10px 20px;
        border-radius: 50px;
        text-decoration: none;
        margin-top: 20px;
        display: inline-block;
    }

    .btn-custom:hover {
        background-color: #3700b3;
    }
</style>
</head>
<body>

    <div class="card">
        <h1>🚀 User Service is Live</h1>
        <p class="mt-3">Welcome to your Spring Boot Application with a modern,
        colorful UI.</p>
        <a href="#" class="btn-custom">Explore Features</a>

```

```
</div>

</body>
</html>
```

pom.xml (Basic)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.1.2</version> <!-- Match with Java 17 compatibility -->
    <relativePath/> <!-- lookup parent from repository -->
  </parent>

  <groupId>com.example</groupId>
  <artifactId>user-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>user-service</name>
  <description>User microservice</description>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- Web -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Optional: Actuator for health checks -->
```

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- Optional: Lombok -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>

<!-- Test (optional) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
</dependencies>

<build>
    <plugins>
        <!-- Spring Boot plugin -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>

```

Dockerfile

```

FROM openjdk:17-jdk-slim
VOLUME /tmp
COPY target/user-service-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]

```

For Helm Template and Value YAMLs

 value.yaml

Know n keep

- Outside world (browser) → 8081 (LoadBalancer port)
- Kubernetes forwards it to → 8080 (inside pod/container)

```
replicaCount: 2

image:
  repository: docker.io/kapilan97/user-service # your DockerHub image
  pullPolicy: IfNotPresent
  tag: latest

service:
  type: LoadBalancer
  port: 8081
  targetPort: 8080 # Spring Boot app runs on 8080

containerPort: 8080

resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 200m
    memory: 256Mi

autoscaling:
  enabled: false

serviceAccount:
  create: true
  name: ""
```

```
ingress:
  enabled: false
  className: ""
  annotations: {}
  hosts:
    - host: user-service.local
      paths:
        - path: /
          pathType: ImplementationSpecific
  tls: []

nodeSelector: {}

tolerations: []

affinity: {}
```

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "user-service.fullname" . }}
  labels:
    app: {{ include "user-service.name" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ include "user-service.name" . }}
  template:
    metadata:
      labels:
        app: {{ include "user-service.name" . }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```



```
imagePullPolicy: {{ .Values.image.pullPolicy }}
ports:
  - containerPort: {{ .Values.containerPort }}
resources:
  limits:
    cpu: {{ .Values.resources.limits.cpu }}
    memory: {{ .Values.resources.limits.memory }}
  requests:
    cpu: {{ .Values.resources.requests.cpu }}
    memory: {{ .Values.resources.requests.memory }}
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "user-service.fullname" . }}
  labels:
    app: {{ include "user-service.name" . }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}
  selector:
    app: {{ include "user-service.name" . }}
```

PHASE 5's Alteration

value.yaml

```
replicaCount: 2

image:
```

```
repository: docker.io/kapilan97/user-service # Your DockerHub repo/image
pullPolicy: IfNotPresent
tag: latest
```

service:

```
  type: LoadBalancer      # Exposes service outside the cluster
  port: 80                  # External port users access (browser-friendly)
  targetPort: 8081         # Internal container port your app listens on
```

```
containerPort: 8081      # Must match your Spring Boot app's port
```

resources:

```
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 200m
    memory: 256Mi
```

autoscaling:

```
  enabled: false
```

serviceAccount:

```
  create: true
  name: ""
```

ingress:

```
  enabled: false
  className: ""
  annotations: {}
  hosts:
    - host: user-service.local
      paths:
        - path: /
          pathType: ImplementationSpecific
  tls: []
```

```
nodeSelector: {}
```

```
tolerations: []
```

```
affinity: {}
```

Here's your **completely updated** `values.yaml` – correctly aligned for your Spring Boot app running on port **8081**, using a DockerHub image, LoadBalancer service,

 `service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "user-service.fullname" . }}
  labels:
    app: {{ include "user-service.name" . }}
spec:
  type: {{ .Values.service.type }}
  selector:
    app: {{ include "user-service.name" . }}
  ports:
    - name: http
      protocol: TCP
      port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}
```

- `type: {{ .Values.service.type }}` → Reads LoadBalancer from `values.yaml`.
- `port: 80`, `targetPort: 8081` → Pulled from `values.yaml` too.
- Clean structure, Helm-compatible.